Abstraction in C

CSC 230 : C and Software Tools

NC State Department of Computer Science

Lecture Topics

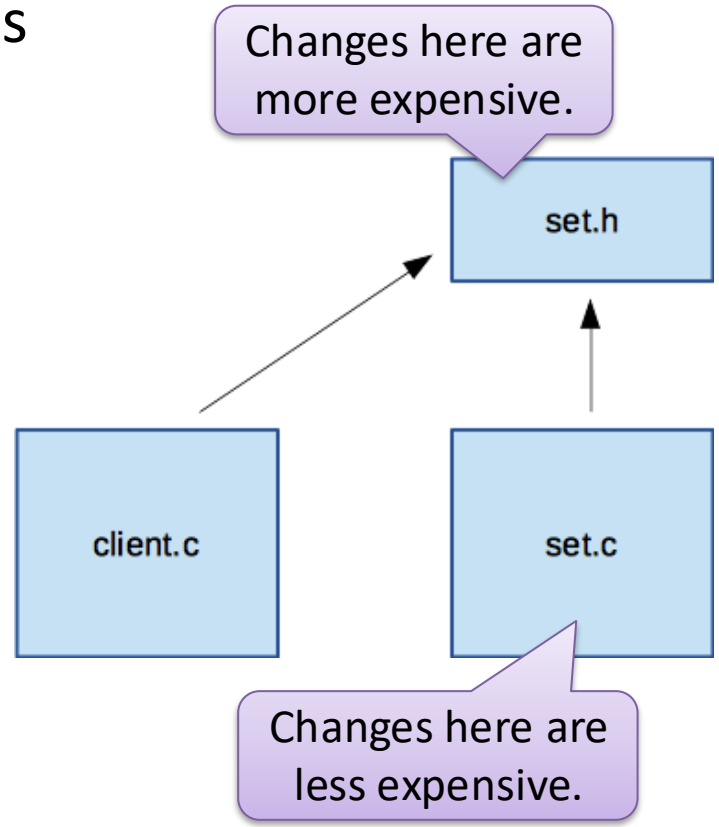
- Designing a component
  - For an abstract object
  - For an abstract data type
  - For a generic abstract data type
- Implementing objects
  - Inheritance
  - Methods
  - Abstract functions and overloading
  - Abstract destruction

Designing a Component

- We're going to look at a few alternatives for designing and implementing a component.
- It will support a set of (integer) values.
  - makeSet() : to initialize the representation
  - insertValue() : to add a value to the set
  - containsValue() : to report whether a value is in the set.
  - destroySet() : to free any resources used by the set.

Designing a Component

- The set component will have a header to advertise its interface.
- And an implementation file (to implement that interface)
- *Client code* is anything else that uses this component.
- We'd like to put as little as possible in the header.
  - Why? If something's in the header, we can't change it without recompiling (all) the client code.
  - Imagine if someone had to fix a bug in stdio.h!



Hiding the Implementation

```
void makeSet();
void insertValue( int val );
bool containsValue( int val );
void destroySet();
```

set.h

```
#include "set.h"

struct Node {
  int val;
  struct Node *next;
};
static struct Node *head;

void makeSet()
{
   head = NULL;
}

void insertValue( int val )
{
  for ( struct Node *n = head; *n; n = n->next )
    ...
```

set.c

Hidden Representation

We could change this representation without breaking client code.

Client Code

```
#include "set.h"

int main()
{
  makeSet();

  insertValue( 2 );
  insertValue( 3 );
  insertValue( 5 );
  insertValue( 7 );
  insertValue( 11 );

  for ( int i = 0; i <= 12; i++ )
    if ( containsValue( i ) )
      ...

  destroySet();
}
```

Initialize the set

Perform some operations with it.

Clean up when we're done.

Abstract Object

- The textbook would call this an *abstract object*.
  - You can use this object without being sensitive to how it's implemented.
- But, here, we can only have one set.
  - We'd like client code to be able to make as many sets as they want.
  - We need to give client code a type that represents a set.

Exposing the Implementation

set.h

```
struct Node {
  int val;
  struct Node *next;
};

typedef struct {
  struct Node *head;
} Set;

void makeSet( Set *s );
void insertValue( Set *s, int val );
bool containsValue( Set *s, int val );
void destroySet( Set *s );
```

Here, use this type for each set you need.

Every function takes a (pointer to the) set it's supposed to operate on.

set.c

```
#include "set.h"

void makeSet( Set *s )
{
  s->head = NULL;
}

...
```

Client Code

```
#include "set.h"

int main()
{
  Set s;
  makeSet( &s );

  insertValue( &s, 2 );
  insertValue( &s, 3 );
  insertValue( &s, 5 );
  insertValue( &s, 7 );
  insertValue( &s, 11 );

  for ( int i = 0; i <= 12; i++ )
    if ( containsValue( &s, i ) )
      ...

  destroySet( &s );
}
```

Make an instance of set.

Initialize it, passing the address of the uninitialized set we just made.

Perform some operations with it.

Clean up when we're done.

No More Abstraction?

- To let client code make instances of Set, we had to let it see our representation.
- This is bad
  - Maybe our design is super secret (less important)
  - Maybe this makes the header larger, which will slow compilation for all code using it (less important)
  - Maybe we want to be able to change our design or fix our implementation later (more important)
- Can we let client code use the Set type … while keeping its contents secret?

Incomplete Type

- Yes. We can give what's called an *incomplete type*.
  - We can tell the compiler about a struct, without saying what's in it.

```
struct ListStruct;
typedef struct RectangleStruct Rect;
```

There's a structure named this.

There's a structure named this, and here's a type name for it.

- This permits *name-only* use of the struct.
  - You can work with pointers to it.
  - … but you can't do anything that depends on its size or its contents.

Re-Hiding the Implementation

- We can use an incomplete type to insulate client code from the implementation.
- But, client code can only use this type as an *opaque pointer*.
- We'd call this design an *Abstract Data Type* (ADT).
- Now, the set.h header might look like:

```
typedef struct SetStruct Set;

Set *makeSet();
void insertValue( Set *s, int val );
bool containsValue( Set *s, int val );
void destroySet( Set *s );
```

Here, use this type when you need a Set.

But, I'm not telling you anything about it. You have to ask me to make one for you.

Looking Inside

```
#include "set.h"

struct Node {
  int val;
  struct Node *next;
};

struct SetStruct {
  struct Node *head;
};

Set *makeSet()
{
  Set *s = (Set *) malloc( sizeof( Set ) ) ;
  s->head = NULL;
  return s;
}
...
void destroySet( Set *s )
{
  ...

  free( s );
}
```

set.c

In the implementation file, we provide the full type definitions.

I'll have to give you a dynamically allocated instance (so, more expensive than stack allocation).

And, you'll need my help when you're done with this object.

Client Code

```
#include "set.h"

int main()
{
  Set *s = makeSet();

  insertValue( s, 2 );
  insertValue( s, 3 );
  insertValue( s, 5 );
  insertValue( s, 7 );
  insertValue( s, 11 );

  for ( int i = 0; i <= 12; i++ )
    if ( containsValue( s, i ) )
      ...

  destroySet( s );
}
```

Please make one of those Set objects I've heard about.

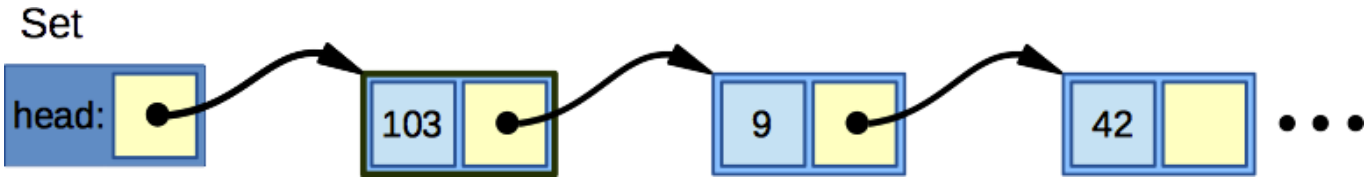
The rest of this is the same, except s is now a pointer.

A Generic, Abstract Data Type

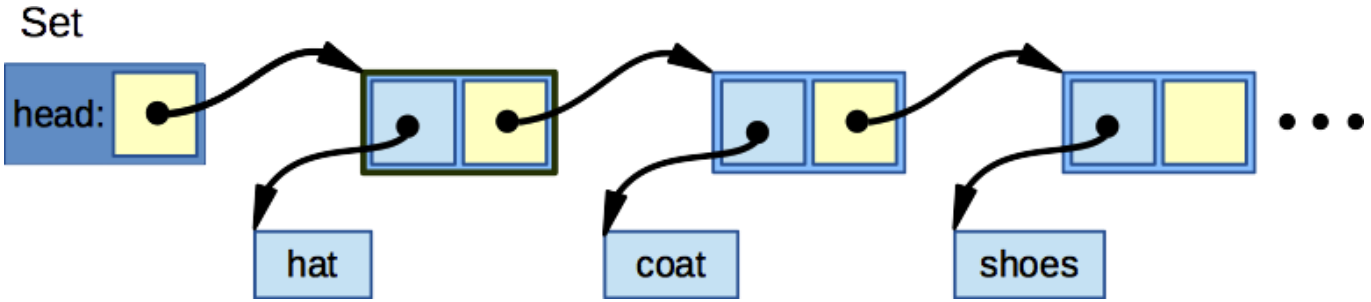
- All we have is a set of ints
- What if we wanted a set of something else?
  - We'd have to make another component
  - … but much of its code would be the same
  - If we were clever, we could make this code generic.
- We would call this a *Generic ADT*

A Generic, Abstract Data Type

- In the implementation, we'd have to be able to store any type of data in our set.
- Previously, we stored a fixed-sized value right inside each node.

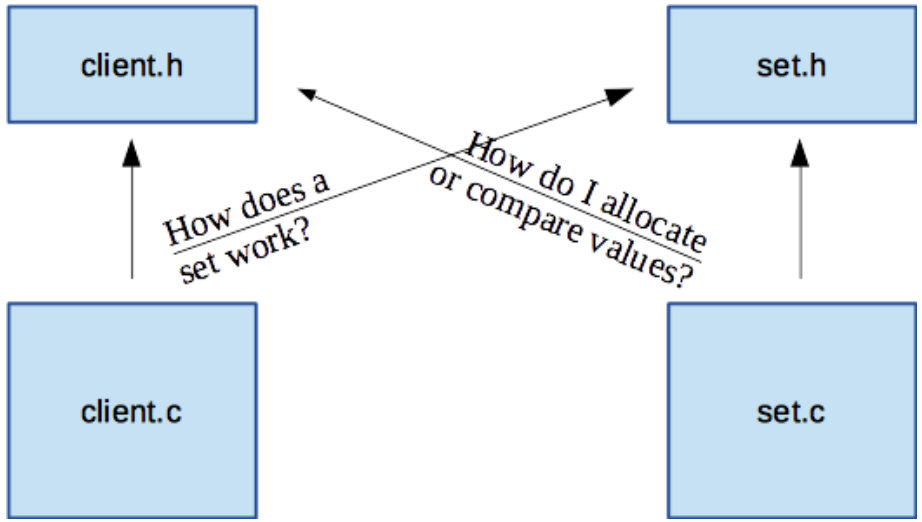


- Now, we'll use a generic pointer to each value, stored elsewhere in memory.



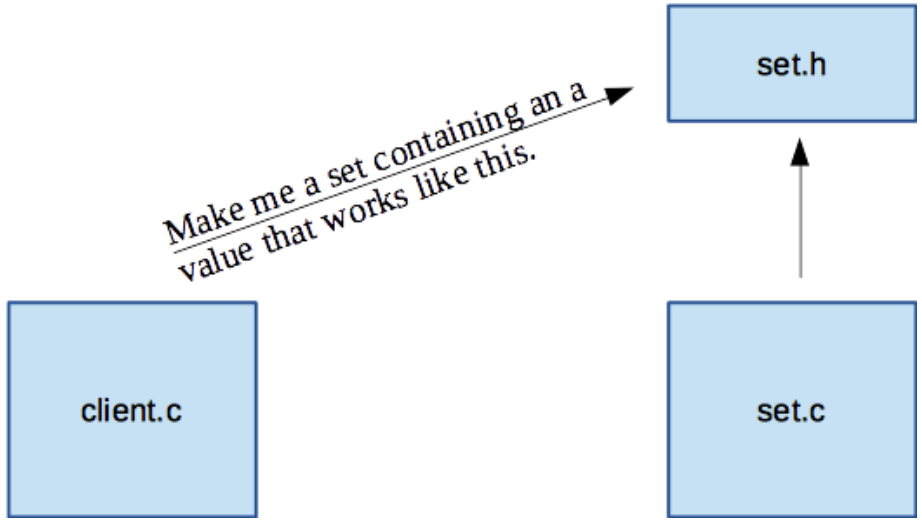
Circular Dependency

- How will our set know
  - What value type it's supposed to contain?
  - Or how to compare values of that type?
- Could it just use code from the client for this?
- No. This is a terrible idea.
- It creates a *circular dependency* in our code.



Avoiding Circular Dependency

- Lots of design cleverness involves eliminating circular dependency.
- Our set needs to provide a mechanism for telling it about the value it's working with.
- Kind of like what qsort() does
- … it provides a way to tell it something about the type of values it's sorting.



Generic Implementation

```
#include "set.h"

struct Node {
  void *val;
  struct Node *next;
};

struct SetStruct {
  size_t vsize;
  bool (*same)(void const *v1, void const *v2);

  struct Node *head;
};

Set *makeSet(size_t vsize, bool (*same)(void const *v1, void const *v2))
{
  Set *s = (Set *) malloc( sizeof( Set ) ) ;
  s->head = NULL;

  s->vsize = vsize;
  s->same = same;

  return s;
}
```

set.c

I could hold any value.

I have to remember how large values are, and how to compare them.

You have to tell me once, when you make each set.

Client Code

```
#include "set.h"

bool sameInt( void const *v1, void const *v2 )
{
  return *(int const *)v1 == *(int const *)v2;
}

int main()
{
  Set *s = makeSet( sizeof( int ), sameInt );

  insertValue( s, (int []){ 2 } );
  insertValue( s, (int []){ 3 } );
  insertValue( s, (int []){ 5 } );
  insertValue( s, (int []){ 7 } );
  insertValue( s, (int []){ 11 } );

  for ( int i = 0; i <= 12; i++ )
    if ( containsValue( s, &i ) )
      ...

  destroySet( s );
}
```

Here's how to compare two ints.

Make a set and tell it how to work with ints.

We have to pass values by address now (using array syntax to get the address of just one literal int).

Object Orientation in C

- You can write object-oriented code in C
  - C programmers used to do this all the time before C++ was created.
- You just have to do a lot of the work yourself
  - This is a little bit painful
  - But worthwhile … it exposes how techniques like inheritance and overriding really work.

An Example of Object-Orientation

- We're going to build classes to represent cells in a spreadsheet.

```
+----------+------------+----------------+
|          |       3.75|             Gum|
+----------+------------+----------------+
|          |       8.90|         Sandwich|
+----------+------------+----------------+
```

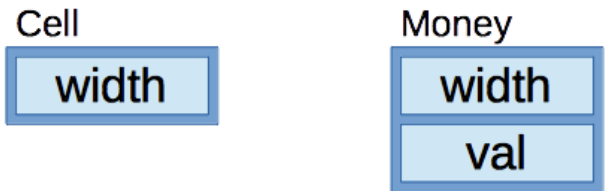
A two-dimensional array of Cell objects.

Different types of cells will store different values.

And print themselves differently.

Inheritance Exposed

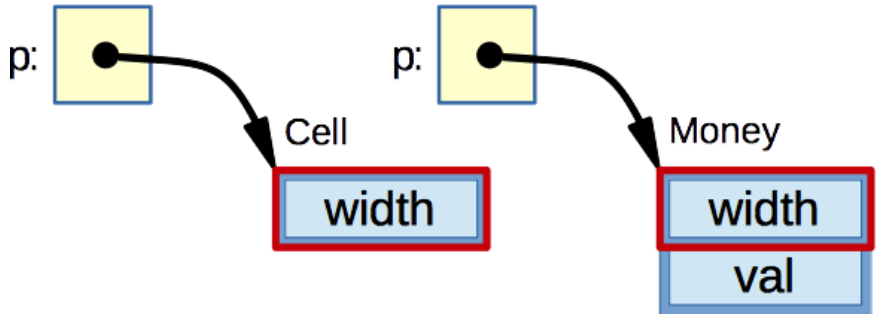
- We'll start with just two types of cells
  - Cell : a superclass that has a width field, but doesn't display anything.
  - Money : a subclass of Cell that also maintains and displays a monetary amount.
- We'll build these so they look like this in memory:



- Money starts out just like Cell
  - But it adds a field later in memory.

Inheritance Exposed

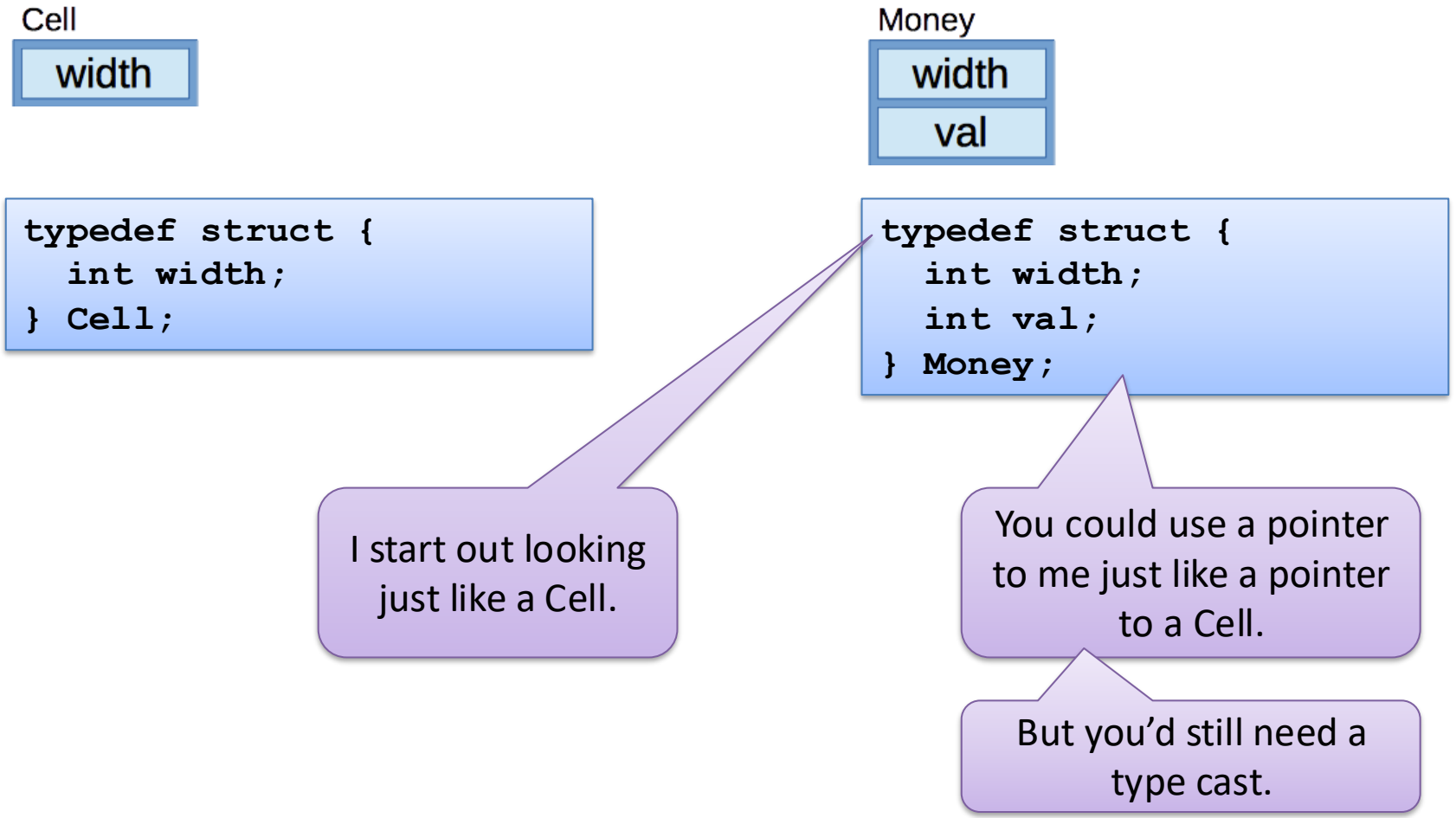
- So, a pointer to a Money instance looks just like a Cell instance.
  - It has all the fields you'd expect in a Cell.
  - In exactly the same layout



- This is how inheritance works.
  - Fields of the superclass are placed at the start of the subclass' fields.
  - With exactly the same layout in memory.

Inheritance In C

- Here's how we could make this work in C

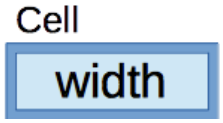


Constructors in C

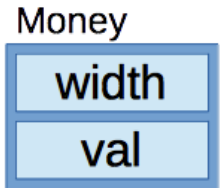
- For a constructor, we just need a function that can:
  - Create each type of object.
  - And return a pointer to it.

```
Cell *makeCell( int width )
{
  Cell *this = (Cell *)malloc( sizeof( Cell ) );
  this->width = width;
  return this;
}
```

Not a reserved word in C, but it helps show what we're doing.




```
Money *makeMoney( int width, int val )
{
  Money *this = (Money *)malloc( sizeof( Money ) );
  this->width = width;
  this->val = val;
  return this;
}
```



Using Objects

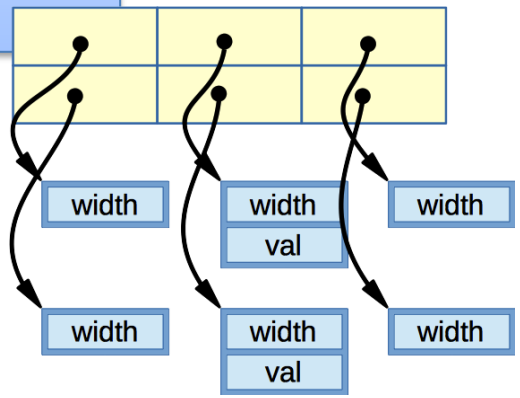
- In client code, we can use a Cell pointer to hold the address of either kind of object.

```
Cell *sheet[ 2 ][ 3 ];

sheet[ 0 ][ 0 ] = makeCell( 10 );
sheet[ 0 ][ 1 ] = (Cell *) makeMoney( 12, 375 );
sheet[ 0 ][ 2 ] = makeCell( 16 );

sheet[ 1 ][ 0 ] = makeCell( 10 );
sheet[ 1 ][ 1 ] = (Cell *) makeMoney( 12, 890 );
sheet[ 1 ][ 2 ] = makeCell( 16 );
```

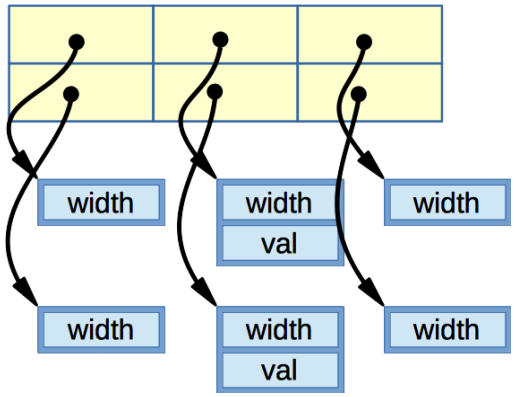
The compiler doesn't know that a Money pointer can be used as a cell pointer.



Using Objects

- In client code, we can expect the same thing from both types of objects, a width field.

```
for ( int i = 0; i < 2; i++ ) {
  for ( int j = 0; j < 3; j++ ) {
    Cell  *c = sheet[ i ][ j ];
    printf( "|%*s", c->width, "" );
  }
  printf( "|\n" );
}
```



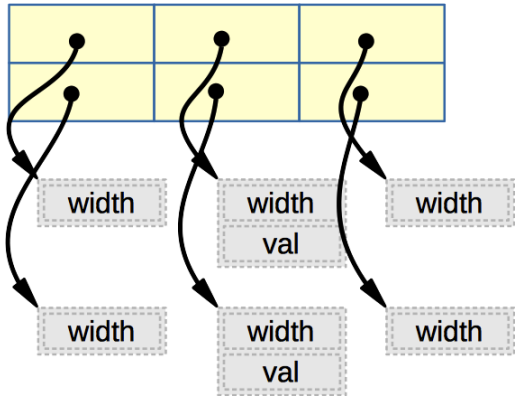

```
+----------+-----------+----------------+
|          |           |                |
+----------+-----------+----------------+
|          |           |                |
+----------+-----------+----------------+
```

With some additional decoration, we get this output.

Cleaning Up

- When we're done with the objects, we can release their memory with free( ).

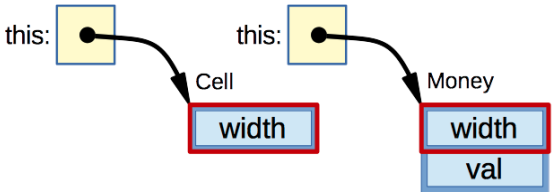
```
for ( int i = 0; i < 2; i++ ) {
  for ( int j = 0; j < 3; j++ ) {
    free( sheet[ i ][ j ] );
  }
}
```



Methods

- A method : function that operates on an instance of an object.
- We can do that. We just need to pass in the instance the function should operate on.
  - Internally, C++ and Java do this also.

```
int getWidth( Cell *this )
{
  return this->width;
}
```

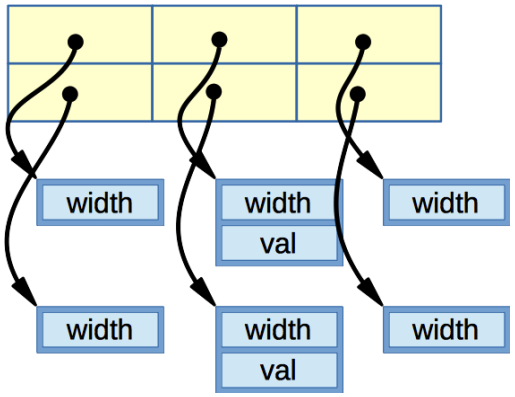


- This function will work for instance of Cell or Money (they both have a width field in the same place)

Calling Methods

- In client code, we can call this method instead of accessing fields directly.

```
for ( int i = 0; i < 2; i++ ) {
   for ( int j = 0; j < 3; j++ ) {
     Cell  *c = sheet[ i ][ j ];
     printf( "|%*s", getWidth( c ), "" );
   }
   printf( "|\n" );
 }
```



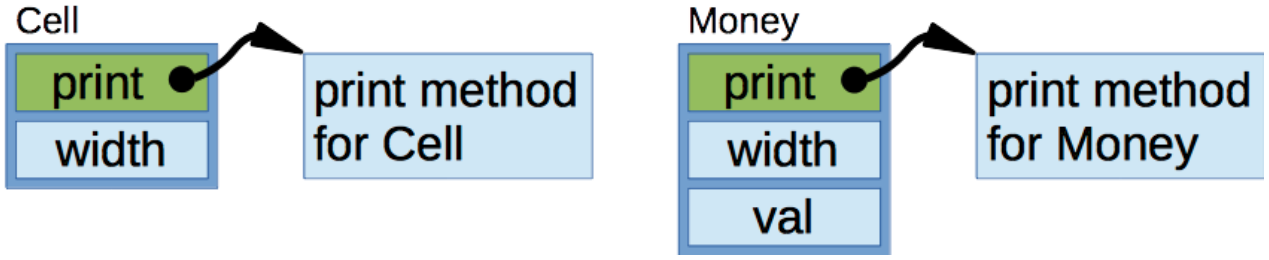
- If we had access restrictions, we could enforce encapsulation.

Overriding Methods

- What if we want these objects to know how to print themselves?
  - That's more interesting.
- Each type of object needs to print itself differently.
  - Cell : just print empty spaces
  - Money : print the val field as dollars and cents
- That sounds like overriding
  - A method that can behave differently in subclasses

Overriding Methods

- We can implement this with function pointers.
- The superclass can have a field for a pointer to the print method.
  - Which subclasses will also have.
- Each instance can carry around a pointer to the print method that's right for that instance.



- In C++, we would call this a *virtual function*.
  - We have to use a function pointer to get the right method.

Overriding Methods in C

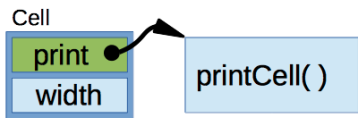
```
typedef struct CellStruct {
  void (*print)( struct CellStruct * );
  int width;
} Cell;

void printCell( Cell *this ) {
  printf( "%*s", this->width, "" );
}

Cell *makeCell( int width )
{
  Cell *this = (Cell *)malloc( sizeof( Cell ) );

  this->print = printCell;

  this->width = width;
  return this;
}
```



Structure for the superclass.

Print method for the superclass.

Store a pointer to the print method in the print field.

Overriding Methods in C

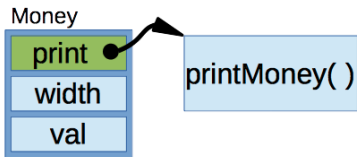
```
typedef struct {
  void (*print)( Cell * );
  int width;
  int val;
} Money;

void printMoney( Cell *cell ) {
  Money *this = (Money *)cell;
  printf( "%*d.%02d", this->width - 3,
          this->val / 100, this->val % 100 );
}

Money *makeMoney( int width, int val )
{
  Money *this = (Money *)malloc( sizeof( Money ) );

  this->print = printMoney;

  this->width = width;
  this->val = val;
  return this;
}
```



Structure for the subclass.

Print method for the subclass.

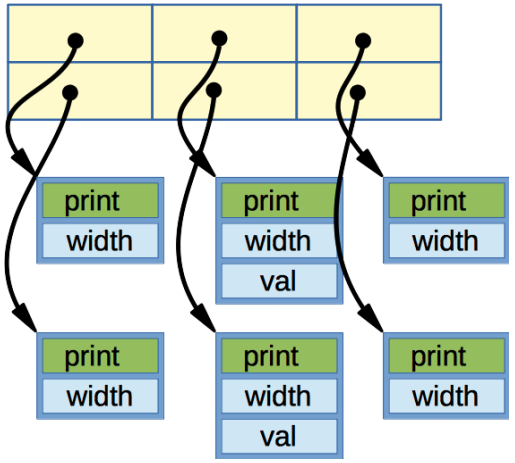
Store a pointer to the print method in the print field.

Calling the Print Method

- To call a virtual function like print, we use the pointer stored in the object's print field.

```
for ( int i = 0; i < 2; i++ ) {
  for ( int j = 0; j < 3; j++ ) {
    printf( "|" );

    Cell *c = sheet[ i ][ j ];
    c->print( c );
  }
  printf( "|\n" );
}
```



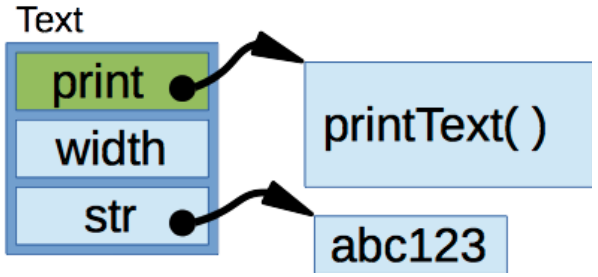

```
+---------+----------+--------------+
|        |    3.75|             |
+---------+----------+--------------+
|        |    8.90|             |
+---------+----------+--------------+
```

Get the function pointer in this instance.

Pass in its this pointer, like any other method.

A More Complex Subclass

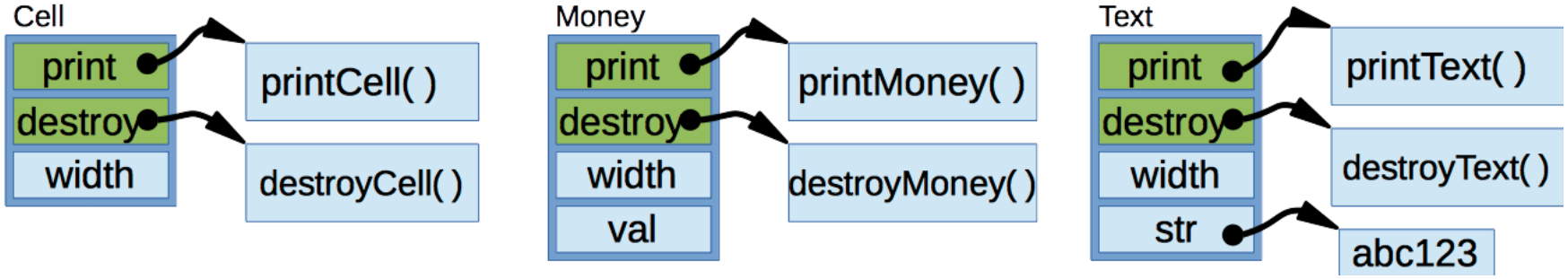
- Let's define another subclass, Text
- Stores an arbitrary, dynamically allocated string field



- Seems straightforward, but it reveals a problem
  - Freeing this object requires special behavior, freeing the dynamically allocated string.
- Solution: Cell and its subclasses should know how to clean up after themselves.

Virtualizing Destruction

- In C++, we will call this a virtual destructor
  - A function that knows how to free memory and other resources allocated by an object
  - The right function to use depends on the type of the object
- We can handle this with another function pointer in the superclass:



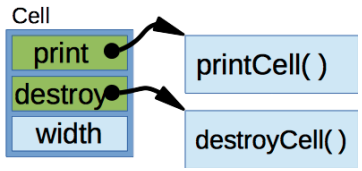
Virtual Destructor

```
typedef struct CellStruct {
  void (*print)( struct CellStruct * );
  void (*destroy)( struct CellStruct * );
  int width;
} Cell;

...

void destroyCell( Cell *this ) {
  free( this );
}

Cell *makeCell( int width )
{
  ...;
  this->destroy = destroyCell;
  ...;
}
```



A new function pointer in the superclass.

A function to destroy a Cell (just free its memory)

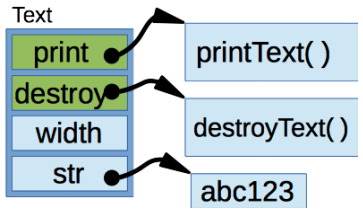
Store a pointer to the destroy method in the destroy field.

Text Objects

```
typedef struct {
  void (*print)( Cell * );
  void (*destroy)( Cell * );
  int width;
  char *str;
} Text;

void printText( Cell *cell ) {
  Text *this = (Text *)cell;
  printf( "%*s", this->width, this->str );
}

void destroyText( Cell *cell ) {
  Text *this = (Text *)cell;
  free( this->str );
  free( this );
}
```



Structure for text, with all the superclass stuff at the start.

A function to print a Text object.

A function to free a text object.

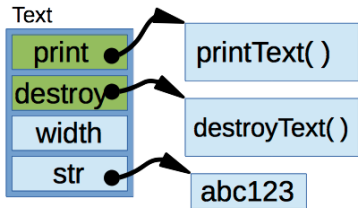
Text Objects

```
Text *makeText( int width, char const *str )
{
  Text *this = (Text *)malloc( sizeof( Text ) );

  this->print = printText;
  this->destroy = destroyText;

  this->width = width;
  this->str = (char *)malloc( strlen(str) + 1 );
  strcpy( this->str, str );

  return this;
}
```



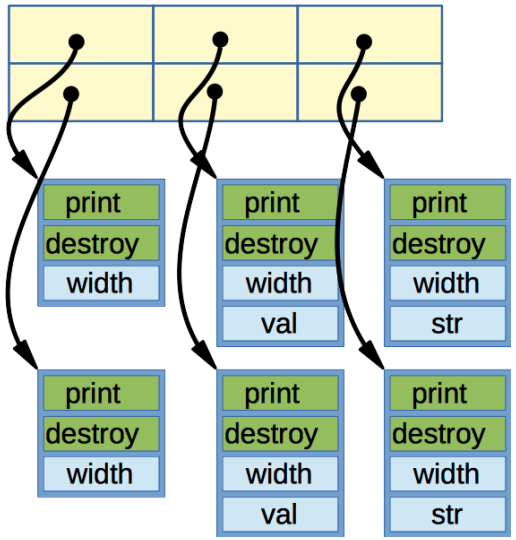
A function to create a text object.

Printing the Cells

- The same client code will print this new type of cell.

```
for ( int i = 0; i < 2; i++ ) {
  for ( int j = 0; j < 3; j++ ) {
    printf( "|" );

    Cell *c = sheet[ i ][ j ];
    c->print( c );
  }
  printf( "|\n" );
}
```




```
+---------+----------+--------------+
|         |     3.75|          Gum|
+---------+----------+--------------+
|         |     8.90|      Sandwich|
+---------+----------+--------------+
```

Here, we added some text cells.

Freeing Cells

- And, each type of cell knows how to destroy itself

```
for ( int i = 0; i < 2; i++ ) {
  for ( int j = 0; j < 3; j++ ) {
    Cell *c = sheet[ i ][ j ];
    c->destroy( c );
  }
}
```

